AI-Driven Quality Assistance Application for STLC Automation

A PROJECT REPORT
SUBMITTED IN PARTIAL FULFILMENT OF THE
REQUIREMENTS FOR THE DEGREE OF

Master of Technology (Online)
IN
Data Sciences and Business Analytics

BY

Monika Tyagi



Faculty of Engineering
Indian Institute of Science
Bangalore – 560 012 (INDIA)

November, 2026

Declaration of Originality

I, **Monika Tyagi**, with SR No. **13-19-02-19-52-24-1-24515** hereby declare that the material presented in the project report titled

AI-Driven Quality Assistance Application
for STLC Automation

represents original work carried out by me at **Alight Services India Pvt Ltd** as part of the project credit requirements of the **Master of Technology (Online)** degree in **Data Science and Business Analytics** at the Indian Institute of Science, between **August, 2024** to **November, 2026**.

With my signature, I certify that:

- I have not manipulated any of the data or results.
- I have not committed any plagiarism of intellectual property. I have clearly indicated and referenced the contributions of others.
- I have explicitly acknowledged all collaborative research and discussions.
- I have understood that any false claim will result in severe disciplinary action.
- I have understood that the work may be screened for any form of academic misconduct.

Date: Student Signature

In our capacities as internal project guide and faculty mentor of the above-mentioned work, we certify that the above statements are true to the best of our knowledge, and we have carried out due diligence to ensure the originality of the report.

Internal Guide Name: *Kapil Madaan* Internal Guide Signature

Organisation: *Alight Services India Pvt Ltd*

Faculty Mentor Name: *Punit Rathore* Faculty Mentor Signature

Department: *Cyber Physical Systems*

© Monika Tyagi

November, 2026

All rights reserved

Acknowledgements

I would like to thank Mr. Kapil Madaan, my internal project guide, for the time he spent discussing the problem and for keeping the work grounded in how quality teams actually operate. I am equally grateful to Prof. Punit Rathore, my faculty mentor, for his questions on scope, evaluation, and how to write this report honestly.

Thanks are also due to the faculty and staff of the Indian Institute of Science. The M.Tech. (Online) programme made it possible to carry this project alongside professional work, and the review process helped me see where the prototype still needed clearer limits.

Abstract

Quality work is rarely confined to one tool. A requirement may live in Jira, the related code change in GitHub, and the latest test run in a log file or a spreadsheet. Each time a tester moves from one of these places to the next, the same story has to be restated: what should the software do, what changed, and what still needs checking. That hand-off is a large part of day-to-day Software Testing Life Cycle (STLC) work, and it is still mostly manual.

This project grew out of that friction. I wanted to see whether an AI assistant could keep the thread of a testing task together—from requirements and stories through diffs, failures, and release notes—without taking the decision away from the engineer. The result is a working prototype: a React and TypeScript frontend, a FastAPI backend with PostgreSQL, and a Google Agent Development Kit (ADK) service. A user submits context; a root agent sends the request to one or more of nine specialists. Those specialists draft test cases and data, comment on a code change, structure a defect, or weigh release risk. Jira Cloud and GitHub can be connected when credentials are available. Microsoft Entra sign-on and several model providers are optional.

I describe the design as it stands in the repository, the related work that shaped it, and how it should be evaluated. The application is a drafting aid, not an autonomous tester. It does not run a team's suite, and a self-healing suggestion is not written back to source. I did not have a comparative benchmark or a measured productivity study for this report. Whether the assistant actually improves traceability, or simply moves effort from writing to reviewing, is still an open question.

Contents

Acknowledgements i

Abstract ii

Contents iii

List of Figures vi

List of Tables vii

List of Algorithms viii

Acronyms, Abbreviations and Symbols x

1 Introduction 1

1.1 Background . . . 1

1.2 Problem Statement and Motivation . . . 2

1.3 Research Questions . . . 2

1.4 Objectives . . . 2

1.5 Scope and Contributions . . . 3

1.6 Report Organization . . . 3

2 Literature Review 4

2.1 Software Testing Life Cycle . . . 4

2.2 LLMs in Software Quality Engineering . . . 4

2.3 Agent-Orchestrated Workflows . . . 5
2.4 Grounding, Traceability, and Human Review . . . 5
2.5 Evaluation of AI-Assisted Testing . . . 6
2.6 Research Gap and Project Position . . . 6

3 Methodology and Implementation **7**
3.1 Development Method . . . 7
3.2 Requirements and Actors . . . 7
3.3 System Architecture . . . 8
3.3.1 Frontend . . . 9
3.3.2 Backend and Persistence . . . 9
3.3.3 Agent Orchestration . . . 10
3.3.4 External Integrations and Model Providers . . . 12
3.4 Security and Data Handling . . . 12
3.5 Deployment and Operations . . . 13
3.6 Quality Attributes . . . 13

4 Results and Discussions **15**
4.1 Prototype Implementation Status . . . 15
4.2 Existing Automated Test Assets . . . 15
4.3 Demonstration Scenarios . . . 15
4.3.1 Interface Walkthrough and Illustrative Outputs . . . 17
4.4 Evaluation Plan . . . 22
4.4.1 Dataset and Protocol . . . 22
4.4.2 Measures . . . 23
4.4.3 Analysis and Validity . . . 23
4.5 Results and Discussion . . . 24
4.6 Limitations . . . 24

5 Conclusions **26**

5.1 Future Work 26

Appendix **28**

5.A Representative Request and Response Flow 28

5.A Reproducibility Checklist for a Future Evaluation 28

5.A Project Materials 29

References **30**

List of Figures

3.1 Logical application architecture. Optional external integrations and model providers are enabled by configuration. 8

3.2 Proposed AWS deployment architecture for the Quality Assistance application. This target design is not currently deployed by the repository. 9

3.3 Illustrative agentic workflow showing orchestration among the root agent and specialist agents. 11

3.4 Illustrative request sequence through the application services for the deployment configuration shown. 12

4.1 Workspace dashboard used to start a quality-assistance request. 18

4.2 Settings page for configuring Jira, GitHub, and sign-in integrations. Availability depends on administrator configuration and valid credentials. 19

4.3 Example request input containing quality context and a user story before agent processing. 19

4.4 Example generated response with prioritized test-case recommendations. 20

4.5 Example suggested test data presented as part of the generated response. 20

4.6 Example automation notes and test-script recommendations. 21

4.7 Example release-risk and readiness analysis based on the submitted context. . . 22

List of Tables

3.1 Selected functional requirements and corresponding implemented behavior. . . . 8

3.2 Registered specialist agents and the scope described by their instructions. 10

4.1 Implementation status as of November 2026, distinguishing current behavior from planned work. 16

4.2 Proposed evaluation measures; no values are reported because a controlled evaluation has not yet been completed. 23

List of Algorithms

1 Assistance request processing 11

Acronyms, Abbreviations and Symbols

The short forms below appear in more than one chapter.

Acronyms

ADK Agent Development Kit

API Application Programming Interface

CI/CD Continuous Integration and Continuous Delivery

LLM Large Language Model

M.Tech. Master of Technology

MCP Model Context Protocol

QA Quality Assurance

RQ Research Question

STLC Software Testing Life Cycle

UI User Interface

Abbreviations

E2E End-to-end

JWT JSON Web Token

ACRONYMS, ABBREVIATIONS AND SYMBOLS

RAG Retrieval-Augmented Generation

REST Representational State Transfer

Chapter 1

Introduction

1.1 Background

Software quality is not produced by running tests alone. In a typical Software Testing Life Cycle (STLC) a team reads requirements, decides what is in scope, designs cases and data, executes checks, files defects, writes a summary, and then argues—sometimes late—about whether a release is ready. Standards and practitioner syllabi treat these steps as planned, traceable work that should follow risk rather than habit [1, 2].

The difficulty I kept seeing is less about any one of those steps and more about how they sit in different tools. Requirements arrive as tickets. Code changes arrive as diffs. Failures arrive as logs. Someone still has to interpret a story as a test, map a commit onto regression scope, and turn a run into evidence a release manager can use. None of this is mysterious on a single ticket. It becomes expensive when the same interpretation is repeated, slightly differently, every sprint.

Large language models (LLMs) are a reasonable fit for the language-heavy parts of that work: summarizing a requirement, proposing boundary cases, or putting a noisy log into a defect template. They are a poor fit for unsupervised judgement. A model can miss a constraint, invent a field, or sound sure when the input is thin. Any assistant that is actually useful in a quality team has to take context in, keep the task narrow, show what it produced, and leave a person to accept or reject it.

1.2 Problem Statement and Motivation

The gap this project tries to close is a practical one. Many teams already automate individual jobs—a test generator here, a report formatter there—yet still stitch requirements, diffs, results, and release notes together by hand. Analysis, triage, and readiness sit outside the pipeline more often than people admit. Context drops between tools, scenarios are missed, and risk shows up in a late review rather than in the original plan.

I therefore built an orchestrated assistant rather than a single prompt. The user can bring requirements and stories, a source-code diff, or a test-run log. A root agent picks the specialists that fit that mix and returns a draft. The draft is meant to be checked. An engineer still has to read the cases, run tests in the right environment, and decide whether a release goes out.

1.3 Research Questions

The questions below follow the project proposal and the mid-term presentation [3, 4]:

1. **RQ1:** How can a multi-agent AI system coordinate several STLC activities through one quality-assistance interface?

2. **RQ2:** What validation, provenance, and human-review controls are needed before AI-generated testing artifacts can be trusted enough to use?

3. **RQ3:** How should the system be evaluated for its effect on requirement-to-test traceability, quality-engineering effort, coverage, and defect analysis?

1.4 Objectives

The work set out to:

- design and implement a full-stack application in which a user can submit quality context and review an AI-assisted response;

- implement a central agent with specialist capabilities for common STLC tasks;

- persist requests against the signed-in user and keep the application behind authentication;

- connect selected engineering sources, in particular Jira issues and GitHub code changes;
- keep model providers and deployment settings configurable; and
- define an evaluation plan that can be repeated, with human review, without claiming benefits that have not been measured.

1.5 Scope and Contributions

This report describes the application as it exists in the repository in November 2026. The main contribution is a working prototype, not a new theory of agents. A browser interface, a persistent API, and an agent service sit together. Users can paste requirements and stories, attach a diff or an execution log, continue a request in a later turn, and export the reply. If Jira Cloud and GitHub are configured, issue text and diffs can be pulled in rather than typed. Behind that, a root agent coordinates nine registered STLC specialists.

A few limits should be kept in view. The prototype does not run a user's test suite, rewrite scripts in a repository, prove that generated cases are complete, or approve a release. Execution, self-healing, and readiness outputs are suggestions based on what was submitted. Persistent retrieval over test assets, CI/CD execution, formal output validation, and a comparative study are not in the inspected code. They remain future work.

1.6 Report Organization

Chapter 2 reviews the testing lifecycle, LLM-assisted testing, agent orchestration, and the validation issues that follow from them. Chapter 3 covers how the prototype was built: method, architecture, implementation, and security. Chapter 4 records what can actually be shown from the repository and sets out the evaluation needed to answer the research questions. Chapter 5 closes with limitations and the work that should come next.

Chapter 2

Literature Review

2.1 Software Testing Life Cycle

Software testing is a planned set of activities. It is meant to evaluate a product and leave evidence about quality, not merely a pass/fail count at the end of a sprint. The ISO/IEC/IEEE 29119 series sets out concepts, processes, and documents for that work. The ISTQB Foundation syllabus covers the same ground from a practitioner angle: planning, analysis, design, implementation, execution, and completion [1, 2]. In both, traceability is the thread that matters. Test conditions should point back to requirements and risks, and conclusions should rest on what was recorded.

The STLC is also messier than a textbook diagram. A failed run can expose a vague requirement. A defect can force new regression cases. A late code change can throw the original scope out. An assistant that claims to help across several of those phases has to carry context with it. Treating each generated document as a one-off answer would recreate the same fragmentation the tools already have.

2.2 LLMs in Software Quality Engineering

Because LLMs read and write text, they can turn a story into scenarios, suggest data partitions, classify a failure, or summarize a run. That looks attractive precisely because those jobs are repetitive. Attractive is not the same as reliable. A model can skip a constraint that was sitting

in the second paragraph, invent a step that no one asked for, or write with a confidence the evidence does not support.

Amershi and colleagues argue that machine-learning work in software teams needs a human-centered workflow, clear ownership, attention to data quality, and ongoing monitoring [5]. I take that as a design constraint rather than a slogan. In a testing assistant the model output is a candidate. A practitioner should be able to check the assumptions, edit the cases, and throw away a recommendation that is not backed by the input—before any of it enters a test plan or a release call.

2.3 Agent-Orchestrated Workflows

An agent architecture splits work among specialists and lets a coordinator decide who is needed. ReAct looks at the mix of reasoning and action. Surveys of LLM agents treat planning, tools, memory, and coordination as the main design knobs [6, 7]. Specialization makes prompts and responsibilities easier to keep straight. It does not, on its own, make the system correct. Extra model calls also add latency and cost, and they create new ways for the coordinator to send the wrong specialist, or too many of them.

The prototype uses a root agent and a registry of STLC specialists. Routing is guided by the request and the context attached to it. In the code I inspected, that coordination is still mostly instructions and generated text. The design is modular, which helped during implementation, but it is not a workflow engine and it has not been formally verified. Schemas between stages, validation gates, and explicit approval points are the obvious next hardening steps.

2.4 Grounding, Traceability, and Human Review

Retrieval-Augmented Generation (RAG) pairs a language model with documents fetched from an external store. Lewis et al. show how those documents can supply the facts a model would otherwise have to guess [8]. In testing, the same idea would mean generating against approved requirements, policies, API specifications, or old defects rather than against a pasted prompt alone. The proposal and mid-term presentation already flag this as later work. The repository I used for this report does not contain a persistent retrieval index over test assets. The current

prototype uses whatever the user attaches, plus whatever the in-memory agent session still holds.

Grounding is only one piece. An evaluation still has to separate "this case came from that requirement" from "this case is actually correct." Human review is how omissions and unsupported claims get caught. For anything that could affect a release, the workflow should keep the evidence in view, make uncertainty visible, and require a person to say yes.

2.5 Evaluation of AI-Assisted Testing

A fair evaluation needs a baseline, inputs that look like real work, frozen model and prompt settings, and outcomes that can be scored more than once. I would look at requirement-to-test links, scenario coverage, human-rated accuracy, time on comparable tasks, defect-classification quality where labels exist, and end-to-end latency. Those numbers answer different questions. A longer list of generated cases is not better coverage. A faster first draft is not less total effort if review and correction eat the difference.

The NIST AI Risk Management Framework talks about governing, mapping, measuring, and managing AI risk [9]. For this project that translates into recording model and configuration versions, writing down how a reviewer should judge an artifact, keeping suggestions apart from verified results, and treating privacy and operational risk as first-class measures rather than an afterthought.

2.6 Research Gap and Project Position

The proposal and mid-term presentation put the gap this way: individual QA tasks can already be automated, but the work that joins requirements, test design, code changes, failures, and release evidence is still loosely coupled. The prototype answers that with one application and a coordinating agent that can call task-specific capabilities. I am not claiming that agent orchestration is a new research idea. The contribution is the engineering artifact—and its value will depend on whether a later evaluation shows that the coordination is useful, the output is dependable, and the workflow is better than a baseline people can actually describe.

Chapter 3

Methodology and Implementation

3.1 Development Method

I built the prototype in passes rather than from a frozen specification. The first pass was about naming the quality-workflow problem clearly. The second was a service-oriented sketch. After that came the user path, the agent path, and only then a hard look at what evidence would be needed to evaluate either. The proposal already asked for a phased evaluation: public and open-source material first, and only later, if approved, anonymized organizational artifacts. The mid-term presentation captured an earlier snapshot of the prototype and of that evaluation plan [3, 4]. This chapter describes the implementation. Comparative evaluation is still planned.

To write about the current state I went through the frontend, backend, agent service, tests, and deployment notes. Claims below are limited to paths I could actually find. The proposal and presentation explain what the project set out to do; they are not evidence that the planned experiments have been run.

3.2 Requirements and Actors

The main user is a quality engineer, a developer, or someone on a release call who needs structured testing guidance rather than another chat window. After signing in, that person supplies requirements or stories, optionally a diff or a test-run log, and then reads the response. A follow-up can reuse a session identifier. An administrator chooses the model provider and

turns integrations on or off through environment settings.

Requirement	Prototype behavior
Context submission	Accepts requirements, user stories, code diffs, and optional session context through the authenticated assistance API.
Quality assistance	Returns a generated response that can include prioritized cases, test data, automation notes, and release risks.
Specialized analysis	Routes suitable requests to registered STLC agents for test design, execution-log analysis, change impact, defects, summaries, and readiness.
User separation	Associates assistance requests with the authenticated user and persists request inputs and responses in PostgreSQL.
Engineering context	Offers Jira Cloud story retrieval and GitHub pull-request or branch-diff retrieval when configured.
Operational configuration	Supports provider/model selection, containerized local startup, health checks, and production configuration checks.

Table 3.1: Selected functional requirements and corresponding implemented behavior.

3.3 System Architecture

The application has four main parts: the browser interface, the API, the database, and the agent runtime. The React and TypeScript frontend talks to the FastAPI backend. After access is checked, the backend stores the request, builds the agent message, and later stores the reply. A separate service runs the root agent and the specialists through Google ADK and a model abstraction. PostgreSQL holds users, requests, and integration records.

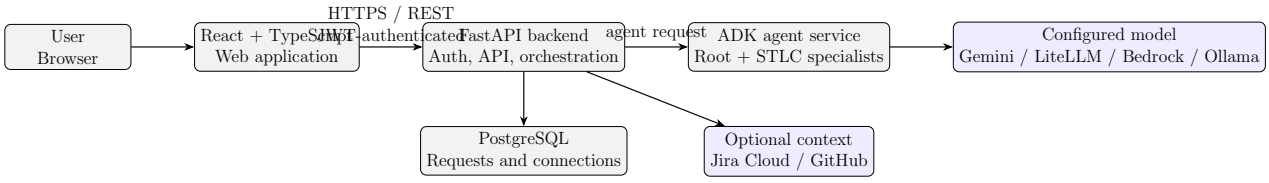


Figure 3.1: Logical application architecture. Optional external integrations and model providers are enabled by configuration.

Figure 3.1 is the software that is actually in the repository. The AWS drawing that follows is a proposed target. It is not running today. The repository has deployment notes, but it does not provision that infrastructure.

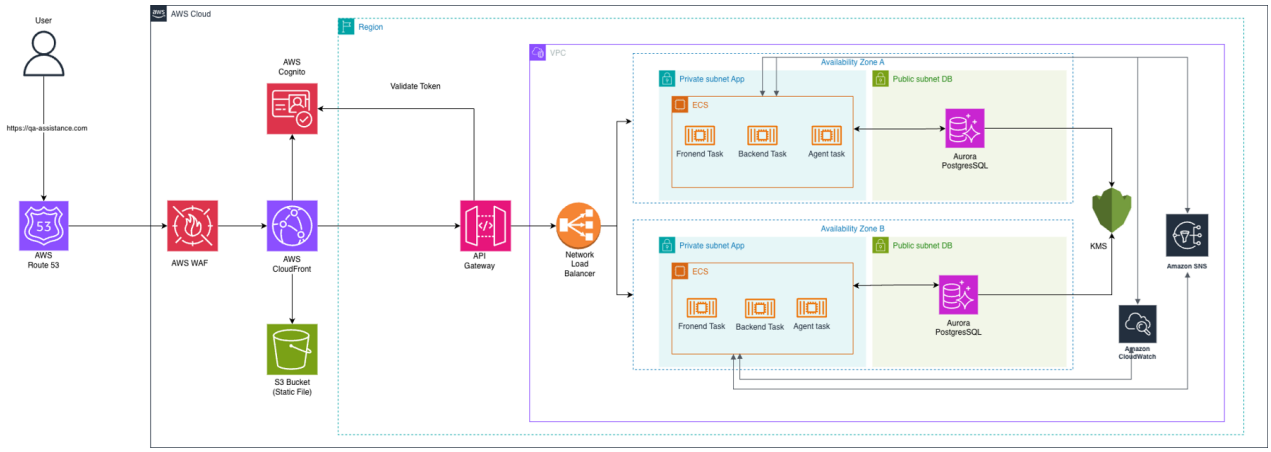


Figure 3.2: Proposed AWS deployment architecture for the Quality Assistance application. This target design is not currently deployed by the repository.

3.3.1 Frontend

The frontend covers sign-in, a dashboard for assistance requests, settings, and a short user guide. On the dashboard a user can type free-form requirements and stories, paste a code change, and start from sample requests: smoke planning, an end-to-end example, change impact, execution and reporting, automation, and a follow-up turn. If Jira is connected, assigned issues can fill the story field. If GitHub is connected, a pull-request or compare diff can be fetched rather than pasted. The reply is shown in the browser and can be downloaded as Markdown or HTML.

3.3.2 Backend and Persistence

The backend is a FastAPI service using asynchronous SQLAlchemy against PostgreSQL [10]. The `/api/assist` route is authenticated. It stores a request with its owner, source text, session identifier, status, timestamps, and, later, the response. It then builds a structured agent message and calls the agent service. Success and failure are both recorded. Errors go back to the frontend through the API's usual error path.

Users, assistance requests, Jira connections, and GitHub repository connections are stored as separate records. That is enough to look up an earlier submission. It is not enough to treat agent conversations as durable history. The current ADK runner keeps session state in memory, so a restart can drop a follow-up context that a user still has on screen.

3.3.3 Agent Orchestration

The root agent is named `quality_assistance_agent`. It is assembled with Google ADK, a configured model, logging callbacks, and the tools that are available at start-up [11]. For the supported cloud providers the root agent can hand work to registered sub-agents. The local Ollama path uses a single, consolidated instruction instead; the specialists are not transferred in that configuration. The backend prompt asks for prioritized test cases, suggested data, automation notes, and release risks, built from the requirements, stories, and diffs that were submitted.

ID	Specialist	Implemented responsibility
S01	Test case generation	Drafts structured, prioritized cases from requirements and stories, including positive, negative, and boundary scenarios.
S02	Test data generation	Suggests representative and edge-case test data.
S03	Test script automation	Suggests automation approaches and repeatable script outlines.
S04	Test execution	Interprets supplied or simulated test-run results and highlights failures; it does not itself launch the user's test suite.
S05	Self-healing test scripts	Recommends maintenance strategies when selectors or interfaces change; it does not directly edit repository files.
S06	Change impact analysis	Relates supplied code diffs to potentially affected areas and regression scope.
S07	Bug report generation	Structures failure information into an actionable defect report.
S08	Test summary reporting	Summarizes test outcomes for stakeholders.
S09	Release readiness evaluation	Weighs supplied test and risk signals and provides a reasoned recommendation.

Table 3.2: Registered specialist agents and the scope described by their instructions.

The router maps requirement and story text to the test-case and test-data agents, diffs to change-impact analysis, execution logs to the execution, defect, and summary agents, and release questions to readiness. There is also a requirement-summarization tool on the root agent. This is prompt-driven routing. There is no separate deterministic workflow engine sitting beside it.

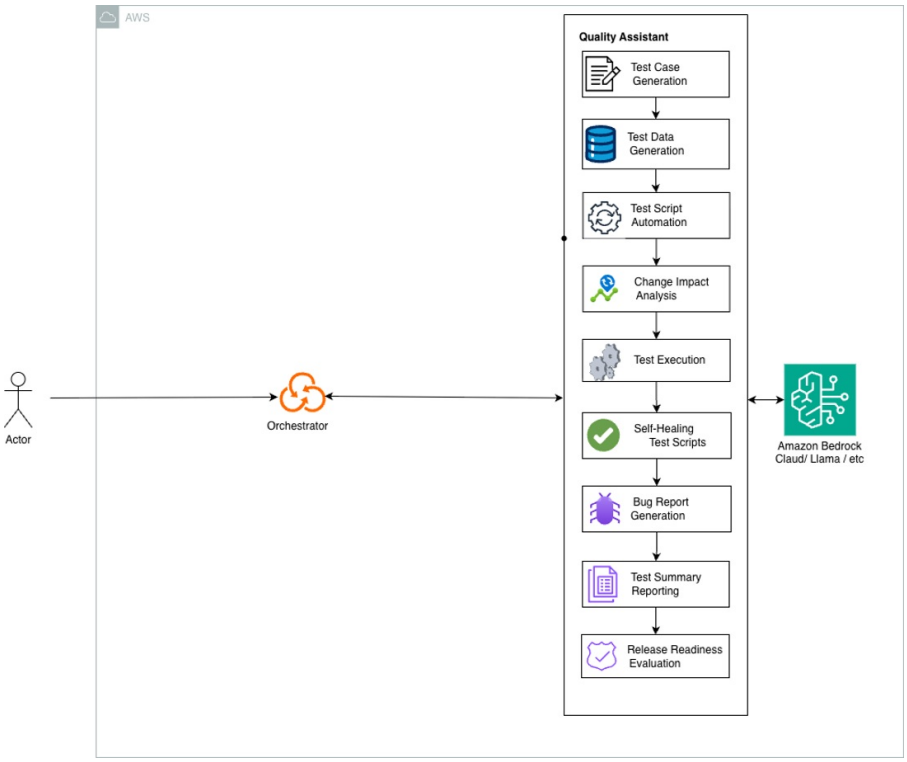


Figure 3.3: Illustrative agentic workflow showing orchestration among the root agent and specialist agents.

Algorithm 1 Assistance request processing

Require: Authenticated user u, requirements r, optional stories s, diff d, session q

Validate the request and associate it with u
Persist request inputs with status `processing`
Construct a message from r, s, and d
Resolve or create an agent session for u
Invoke the root agent with the message and session
Route to one or more specialists according to the request context
Collect the generated response from the agent events
if agent call succeeds **then**
    Persist session identifier, response, and status `completed`
else
    Persist failure status and return an explicit service error
end if
Return the request identifier, session, response, and status

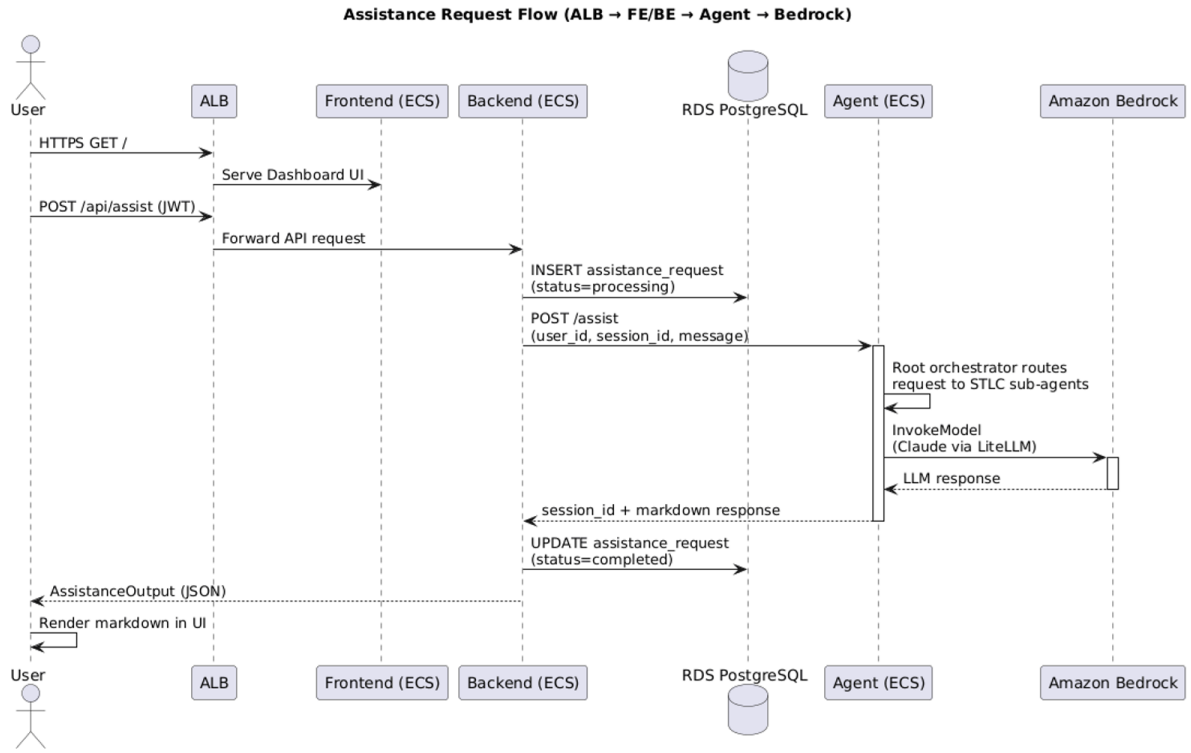


Figure 3.4: Illustrative request sequence through the application services for the deployment configuration shown.

3.3.4 External Integrations and Model Providers

Jira Cloud uses OAuth 2.0, a short-lived state record, and Jira APIs to list a user's assigned issues and to fetch issue detail. GitHub settings store a repository and an encrypted personal access token; the backend can then pull a pull-request diff or compare two references. Microsoft Entra single sign-on is optional. The agent configuration accepts Gemini, LiteLLM, Amazon Bedrock, and Ollama, provided credentials and models are available. Those are options in the code. They are not a claim that every provider was benchmarked on the same tasks.

3.4 Security and Data Handling

Assistance records belong to the signed-in user. JWT access tokens are issued after registration or login. Password handling is delegated to the project's password manager. Jira and GitHub credentials are stored encrypted. Jira OAuth state is a digest and it expires. In production

the settings check looks for real secrets, an encryption key, explicit HTTPS origins, database TLS, and non-default database credentials. The ECS guide describes a pattern with private application and database services, IAM roles, and managed secrets.

Requirements, stories, diffs, and test results can easily contain proprietary information. The application stores them with the request and sends them to whatever model provider is configured. Anyone deploying this has to pick a provider and retention policy that their organization will accept, restrict who can see stored data, rotate credentials, and keep secrets out of the prompt. The controls in the prototype reduce accidental exposure. They do not replace a privacy assessment, a threat model, an independent review, or an internal approval.

3.5 Deployment and Operations

For local work the repository has a Docker Compose stack covering PostgreSQL, frontend, backend, and agent. There are also scripts for a local frontend and Python services against a local or containerized database. The AWS ECS Fargate notes describe a pilot with separate containers, RDS PostgreSQL, an Application Load Balancer, and a chosen model provider. That document is a checklist. It is not infrastructure-as-code, and this report does not claim that an AWS production environment has been stood up.

3.6 Quality Attributes

- **Modularity:** frontend, API, persistence, and agent service are separate processes; specialists are registered on their own.
- **Configurability:** model providers, endpoints, integrations, and production flags come from the environment.
- **Traceability:** the backend stores each user's inputs and responses; a formal requirement-to-case evidence trail and durable agent memory are still missing.
- **Availability:** health and readiness endpoints and container notes help operations; how resilient a real deployment is depends on the topology that is actually chosen.

- **Reliability:** the backend has explicit error paths and settings checks. Generated quality artifacts still need a person to read them, and they still need an empirical evaluation.

Chapter 4

Results and Discussions

4.1 Prototype Implementation Status

What I can show at this stage is a working full-stack prototype, not only an architecture sketch. A user can register, sign in, send context through an authenticated API, and read a response on the dashboard. Requests are stored. The agent service registers nine specialists, and the root agent chooses among them using its routing instructions. The same repository also contains GitHub diff retrieval, Jira Cloud OAuth and issue retrieval, and optional Microsoft Entra authentication.

4.2 Existing Automated Test Assets

There are focused tests for Bedrock credential settings, backend production configuration, Jira Cloud service behavior, and Microsoft Entra SSO. Those files show that a few configuration and integration paths were important enough to get their own checks. They do not tell a reader that the suite currently passes, and they do not cover every UI, model, database, or agent path end to end. A later submission that wants to claim test evidence should include the command, environment, commit, test count, and the actual run outcome.

4.3 Demonstration Scenarios

The dashboard ships with sample inputs that exercise several agent routes:

Project area	**Status in inspected repository**	**Evidence or qualification**
Problem analysis and architecture	Implemented	Research framing is documented in the proposal and presentation; current services are separated into frontend, backend, and agent.
Core quality-assistance flow	Implemented	Authenticated request persistence, prompt construction, agent call, and response persistence are present.
Specialist STLC capabilities	Implemented	Nine task-focused agents are registered; orchestration is controlled by ADK and agent instructions.
Jira and GitHub context	Implemented, configuration-dependent	OAuth and issue retrieval are available for Jira; GitHub repository settings support diff retrieval.
Authentication and deployment controls	Implemented in part	Password/JWT authentication, optional Entra SSO, encrypted integration credentials, and production settings validation are present.
RAG over organizational test assets	Planned	No persistent requirements/test/defect retrieval index is evidenced in the inspected implementation.
Automated CI test execution and script mutation	Planned / out of scope of current behavior	Execution and healing agents analyze or recommend; they do not execute external suites or commit changes.
Comparative empirical evaluation	Pending	No measured baseline, benchmark results, or controlled organizational pilot results were supplied.

Table 4.1: Implementation status as of November 2026, distinguishing current behavior from planned work.

1. a password-reset requirement, used for test planning and requirement summarization;
2. an e-commerce checkout with requirements, user stories, and a code diff;
3. a diff-heavy authentication change, used for regression impact;
4. a test-run log with pass, fail, and skipped cases, used for execution analysis and reporting;
5. a Playwright selector change, used for automation and self-healing notes; and
6. a follow-up that reuses the previous session identifier.

These examples were useful while building and demonstrating the interface. They are not a benchmark. They do not measure coverage, effort, or defect detection.

4.3.1 Interface Walkthrough and Illustrative Outputs

The screenshots follow an ordinary request: open the workspace, glance at integration settings, enter a requirement and a user story, then read the reply. They document the screens and a sample output. No controlled evaluation sits behind them, and the recommendations still need a person to review them.

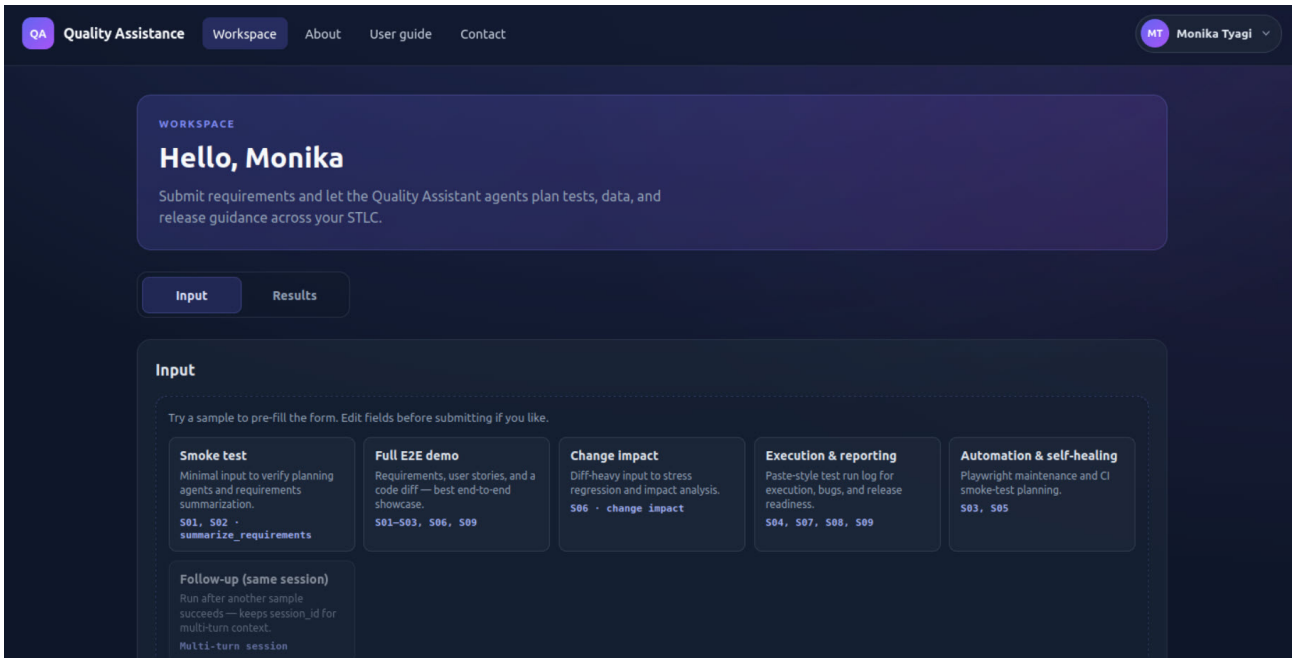


Figure 4.1: Workspace dashboard used to start a quality-assistance request.

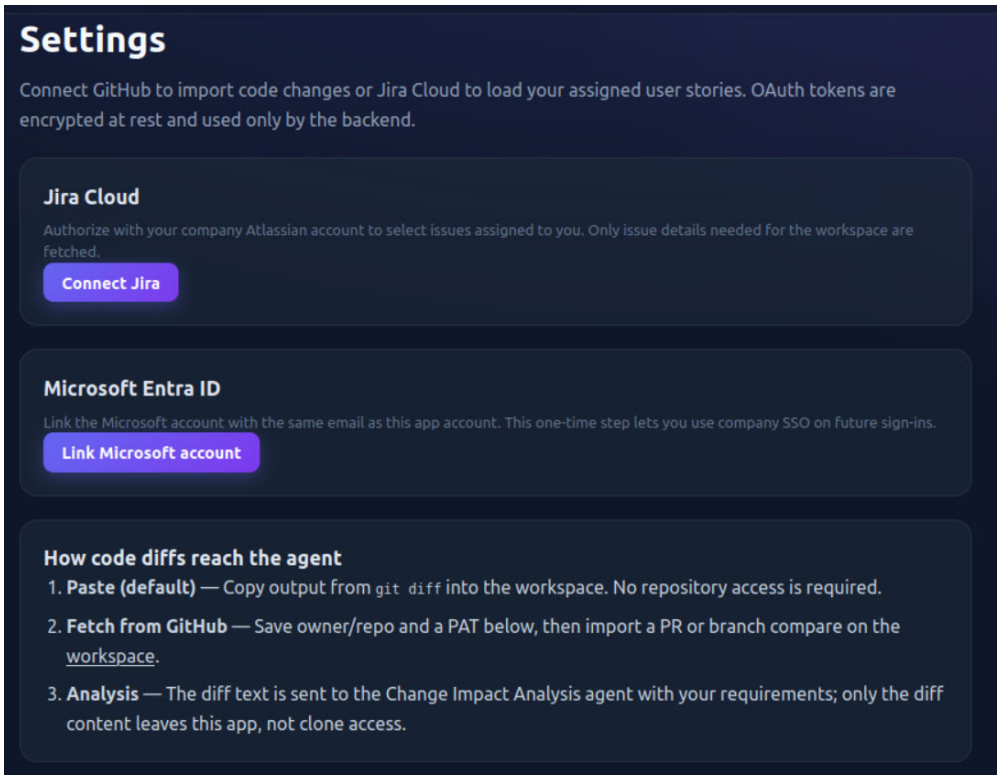


Figure 4.2: Settings page for configuring Jira, GitHub, and sign-in integrations. Availability depends on administrator configuration and valid credentials.

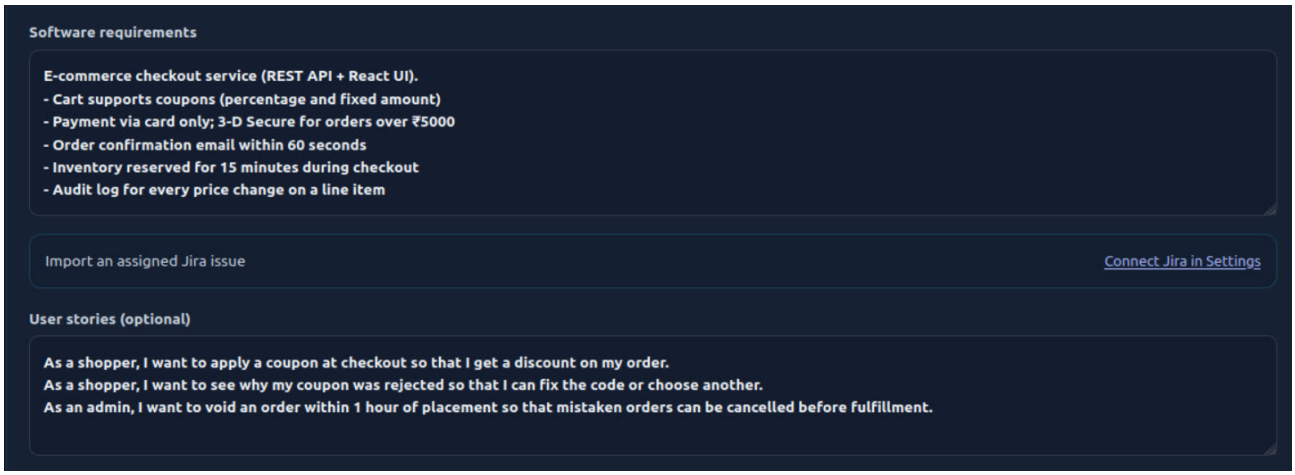


Figure 4.3: Example request input containing quality context and a user story before agent processing.

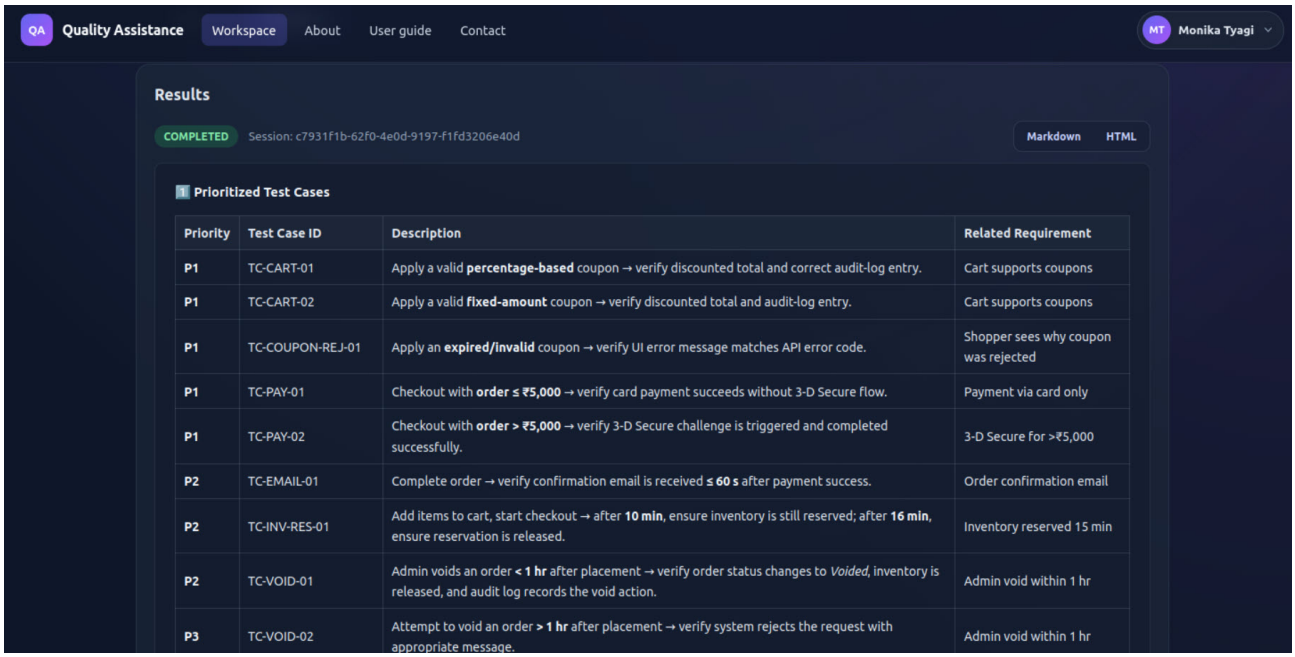
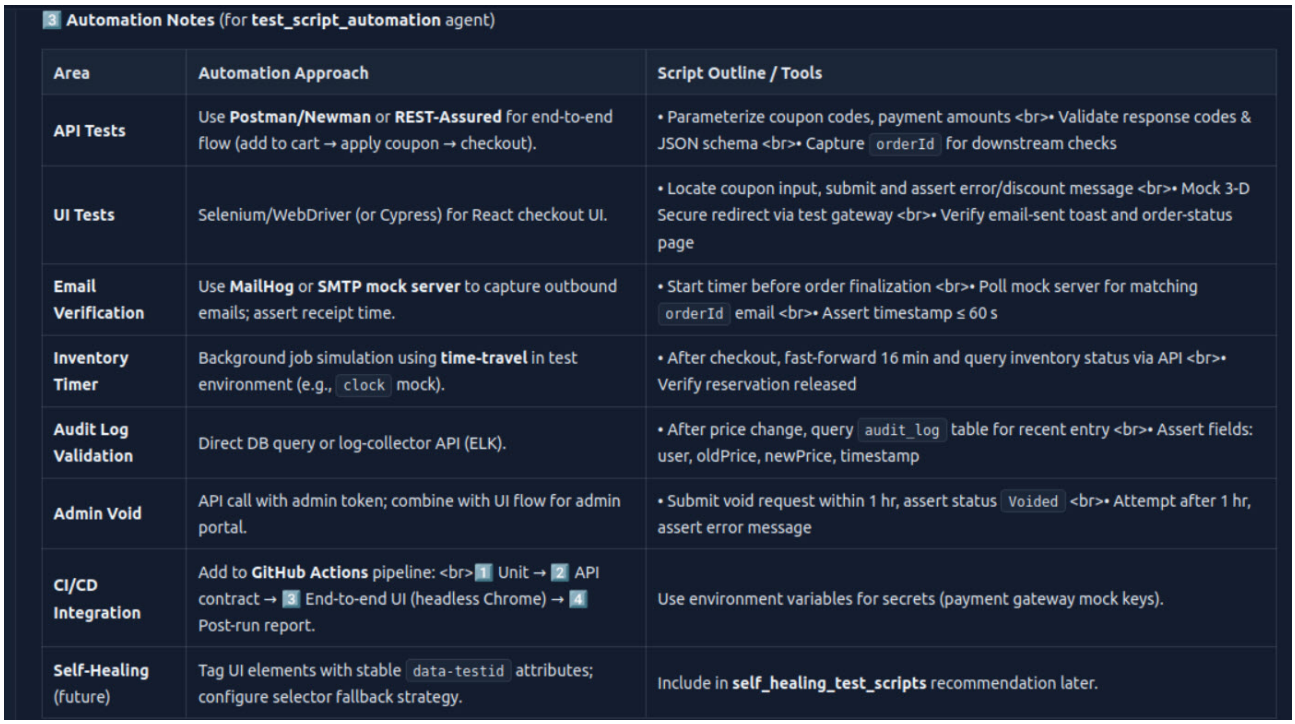


Figure 4.4: Example generated response with prioritized test-case recommendations.



Figure 4.5: Example suggested test data presented as part of the generated response.



3 **Automation Notes** (for **test_script_automation** agent)

Area	Automation Approach	Script Outline / Tools
API Tests	Use **Postman/Newman** or **REST-Assured** for end-to-end flow (add to cart → apply coupon → checkout).	• Parameterize coupon codes, payment amounts • Validate response codes & JSON schema • Capture `orderId` for downstream checks
UI Tests	Selenium/WebDriver (or Cypress) for React checkout UI.	• Locate coupon input, submit and assert error/discount message • Mock 3-D Secure redirect via test gateway • Verify email-sent toast and order-status page
Email Verification	Use **MailHog** or **SMTP mock server** to capture outbound emails; assert receipt time.	• Start timer before order finalization • Poll mock server for matching `orderId` email • Assert timestamp ≤ 60 s
Inventory Timer	Background job simulation using **time-travel** in test environment (e.g., `clock` mock).	• After checkout, fast-forward 16 min and query inventory status via API • Verify reservation released
Audit Log Validation	Direct DB query or log-collector API (ELK).	• After price change, query `audit_log` table for recent entry • Assert fields: user, oldPrice, newPrice, timestamp
Admin Void	API call with admin token; combine with UI flow for admin portal.	• Submit void request within 1 hr, assert status `Voided` • Attempt after 1 hr, assert error message
CI/CD Integration	Add to **GitHub Actions** pipeline: 1 Unit → 2 API contract → 3 End-to-end UI (headless Chrome) → 4 Post-run report.	Use environment variables for secrets (payment gateway mock keys).
Self-Healing (future)	Tag UI elements with stable `data-testid` attributes; configure selector fallback strategy.	Include in **self_healing_test_scripts** recommendation later.

Figure 4.6: Example automation notes and test-script recommendations.

4️⃣ **Release Risks & Mitigations** (for **release_readiness_evaluation**)

Risk	Impact	Likelihood	Mitigation
Coupon mis-calculation (percentage vs fixed)	Wrong discount → revenue loss, poor UX	Medium	End-to-end price verification in every build; unit tests for discount engine
3-D Secure failure for >₹5,000 orders	Payment decline, cart abandonment	High (depends on external gateway)	Mock gateway in staging; monitor real-time success rate after release
Email latency >60 s	Customer confusion, support tickets	Medium	Deploy email service with SLA; add alert on queue depth; include performance test in CI
Inventory reservation timeout bug	Overselling, fulfillment issues	Low to Medium	Automated timer test; log monitoring for "reservation expired" events
Audit log omission	Compliance breach, debugging difficulty	Low	Enforce log write via middleware; unit test audit interceptor
Admin void window logic error	Orders stuck in pending state, financial reversal problems	Medium	Regression suite covering <1 hr and >1 hr cases; code review of time-window logic
Concurrency coupon application	Race condition causing double discount	Low	Stress test with parallel users; lock cart updates at DB level
React UI regression (broken data-testid)	Automation scripts break, manual testing overhead	Medium	Adopt `data-testid` convention; run UI regression smoke after each UI change

Overall Release Readiness – If all **P1** test cases pass in CI, email latency ≤ 45 s in load test, and audit-log coverage is 100 %, the release can be considered **green**. Any failure in P1 or a missed SLA (email, 3-D Secure) should halt promotion to production.

Figure 4.7: Example release-risk and readiness analysis based on the submitted context.

4.4 Evaluation Plan

A later evaluation should put three conditions on the same tasks: a manual QA baseline, an existing point-automation workflow where the team already has one, and the assistant-supported workflow. The first phase should stay on public requirements and open-source repositories or issue trackers, as the project plan already proposed. Any later organizational study should use anonymized artifacts and should wait for the data-governance approval that the organization requires.

4.4.1 Dataset and Protocol

For each project, pick a bounded set of requirements, user stories, code changes, and any test-run or defect records that already exist. Write down the expected requirement-to-test links and expert-reviewed reference outputs before the assistant is run. Freeze the model, model version,

prompt revision, and application configuration for a given comparison. Because model output moves around, repeat tasks where it is practical and keep both the raw output and the review decision. Where it is feasible, the reviewer rating the artifacts should not know which workflow produced them.

4.4.2 Measures

Measure	Operational definition	Interpretation
Requirement traceability	Percentage of in-scope requirements linked to at least one reviewer-accepted test case.	Tests whether generated artifacts preserve requirement coverage links.
Scenario coverage	Reviewer-rated coverage of agreed positive, negative, boundary, and risk conditions.	Measures breadth and relevance rather than the raw number of generated cases.
Artifact correctness	Proportion of sampled cases judged correct and actionable by independent reviewers.	Captures factual and procedural quality; report agreement between reviewers.
Effort	Active authoring and review time per requirement or change.	A reduction is meaningful only if review and correction time are included.
Defect analysis quality	Precision, recall, or F-score against adjudicated defect categories when labels exist.	Applies only where a suitable labeled reference set is available.
Response time and cost	End-to-end latency and model usage per completed task.	Quantifies operational overhead and helps assess scalability.
User acceptance	Structured ratings of usefulness, clarity, and trust from representative practitioners.	Complements objective measures but should not replace correctness review.

Table 4.2: Proposed evaluation measures; no values are reported because a controlled evaluation has not yet been completed.

4.4.3 Analysis and Validity

Quantitative comparisons should report sample sizes, distributions, confidence intervals, and the tests that were used. Error categories matter as much as averages. Qualitative review should look for omissions, unsupported statements, muddy requirements, and places where the model and an expert disagree. Model and prompt versions have to be recorded, otherwise a later rerun is hard to interpret.

The usual threats apply: datasets that do not look like the work people actually do, reviewers who disagree, models that do not give the same answer twice, provider-side changes, examples that leak into prompts, and generated cases that simply create more review work. A small public-data study would be enough to talk about feasibility. It would not, by itself, support claims about enterprise workflows.

4.5 Results and Discussion

The evidence I have for this chapter is implementation evidence. The application takes quality context, stores a request, calls the orchestrated agent, and returns a response. The specialist registry and the dashboard samples cover several STLC tasks. The integration code is a starting point for connecting the engineering systems a team already uses.

I did not have experiment results, a benchmark set, time-saved numbers, coverage figures, defect-detection rates, or a user study to put next to that software. I therefore cannot claim that the system improves quality or reduces effort. What this stage produced is the artifact and a protocol for testing those claims later. A plausible-looking generated case is not proof that the case is complete, correct, or good at finding defects.

4.6 Limitations

- Recommendation quality follows the completeness of the submitted context and the behaviour of the chosen model.
- Routing and synthesis between agents are still mostly prompt-driven. Formal schemas between stages and independent quality gates are not in the inspected code.
- Agent session state lives in an in-memory runner. It is not a durable conversation store across restarts.
- The system does not run external test suites, edit scripts, or enforce a release approval.
- Persistent retrieval over requirements, test assets, and historical defects, and a Jenkins or equivalent CI/CD hook, remain future work.

- Repository tests cover selected settings and services. UI, model-quality, load, and full end-to-end tests are still needed.
- Production deployment, and organization-specific security and privacy approval, are not demonstrated by the deployment planning notes.

Chapter 5

Conclusions

I set out to join several STLC tasks in one place, and the repository now has a full-stack application that does that in draft form. The React and TypeScript frontend, the FastAPI and PostgreSQL backend, and the Google ADK agent service meet at a root agent that can send work to nine specialists. Those specialists cover test design and data, change impact, defect reports, and release-readiness notes. Authentication, request storage, Jira Cloud and GitHub context, optional Microsoft Entra SSO, and a choice of model providers are also in place.

Connecting those pieces is a useful engineering result. It is not yet a result about quality. I have not shown, in a controlled comparison, that the assistant improves coverage, reduces the total time a quality engineer spends, or finds more defects. The agents write suggestions for people to judge. Execution analysis does not run a suite. A self-healing note does not change code. A readiness paragraph does not approve a release.

5.1 Future Work

The next phase should, in my view, concentrate on:

1. structured outputs and explicit checks between specialist stages;
2. human approval and edit points for test cases and for anything that could affect a release;
3. an evaluation of requirement-to-test traceability, correctness, review effort, latency, and cost against a baseline that is written down in advance;

4. trials on public projects and, after approval, on anonymized organizational artifacts;
5. retrieval over approved requirements, test repositories, and historical defects;
6. a careful CI/CD hook for test execution, together with durable, auditable session history; and
7. load, reliability, accessibility, privacy, and security reviews before anyone treats this as a broader deployment.

The interesting question is no longer whether an LLM can produce a testing artifact that looks plausible. It is whether an orchestrated assistant can produce useful, traceable work often enough that the review it creates is still cheaper than the work it replaced. The prototype is a place to ask that question. The answer will have to come from a comparison against a baseline, not from the screenshots of a working demo.

Appendix

5.A Representative Request and Response Flow

A typical request starts with a signed-in user entering some mix of requirements, user stories, and a code diff. The frontend posts that context to the backend assistance route. The backend ties the request to the current user, saves it with a processing status, and builds an agent message. The agent service reuses an existing session or opens a new one, calls the root agent, and collects text from the agent events. The backend then stores the reply and the final status and returns the request and session identifiers. If the user wants to continue, the same session identifier goes back with the next turn.

5.A Reproducibility Checklist for a Future Evaluation

- Record the application revision, model provider, model identifier, prompt revision, and runtime configuration.
- Keep anonymized inputs and the reference cases that will be used for scoring.
- Use the same task set and the same review rubric for every baseline and assistant condition.
- Time generation separately from human editing and review.
- Track accepted, edited, and rejected items, and why.
- Report aggregate scores and the failures; do not drop unsafe or misleading outputs.
- Get approval before using organizational requirements, source code, or defect records.

5.A Project Materials

The project proposal records the motivation, the questions, the phased evaluation, and the work plan. The mid-term presentation describes the agent framework that was intended and the status at that point. Implementation and deployment notes in the repository are the source for the as-built architecture and for the feature limits stated in this report.

References

[1] International Organization for Standardization. *ISO/IEC/IEEE 29119-1:2022: Software and systems engineering—Software testing—Part 1: General concepts*. ISO, 2022. 1, 4

[2] International Software Testing Qualifications Board. Certified tester foundation level syllabus, version 4.0. https://www.istqb.org/certifications/certified-tester-foundation-level, 2023. 1, 4

[3] Monika Tyagi. Ai-driven quality assistant agent for STLC automation: M.tech. (online) project proposal and work plan, 2026. Unpublished project proposal, Indian Institute of Science. 2, 7

[4] Monika Tyagi. Ai-driven quality assistant: Mid-term project presentation, 2026. Unpublished project presentation, Indian Institute of Science. 2, 7

[5] Saleema Amershi, Andrew Begel, Christian Bird, Robert De Line, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, and Thomas Zimmermann. Software engineering for machine learning: A case study. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice*, pages 291–300, 2019. doi: 10.1109/ICSE-SEIP.2019.00042. 5

[6] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. 5

[7] Lei Wang, Cheng Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan

Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wei Zhao, Zhe Wei, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2024. 5

[8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474, 2020. 5

[9] National Institute of Standards and Technology. Artificial intelligence risk management framework (AI RMF 1.0). Technical Report NIST AI 100-1, U.S. Department of Commerce, 2023. 6

[10] Sebastián Ramírez. Fastapi documentation. https://fastapi.tiangolo.com/. Accessed November 2026. 9

[11] Google. Agent development kit documentation. https://google.github.io/adk-docs/. Accessed November 2026. 10